

Anàlisi i Recomanacions per a un Sistema d'Empaquetament 3D Robust en Python

Aquest informe de recerca profunditza en el problema d'empaquetament 3D bin packing que l'usuari està experimentant, oferint una anàlisi exhaustiva dels seus problemes actuals i proporcionant un conjunt de recomanacions pràctiques. L'objectiu és oferir una ruta clara cap a la creació d'un sistema robust, precís i automàtic, utilitzant tecnologies de codi obert. Aquest document aborda les àrees crítiques de detecció de col·lisions, gestió de rotacions, selecció d'algoritmes heurístics, integració amb biblioteques Python i estratègies avançades, tot basant-se estrictament en la informació proporcionada. Es pren com a punt de partida l'extracció de codi proporcionada, que revela els punts febles concrets de l'enfocament actual.

Millora de la Detecció de Col·lisions: De l'AABB a la Intersecció Precisa

El principal punt feble detectat en l'enfocament actual és la ineficàcia o absència de detecció de col·lisions realment precisa entre models 3D complexos. El mètode `_simple_grid_packing`, que simplement calcula posicions en una graella regular, no verifica si una peça es superposa amb cap altra ja empaquetada o amb les parets del contenidor. Aquesta fallada és fonamental perquè l'empaquetament 3D correcte depèn de poder respondre a la pregunta "pot aquest objecte anar aquí sense tocar res?". Per tant, millorar la detecció de col·lisions és el primer pas cap a qualsevol solució viable. L'anàlisi de les fonts disponibles revela una jerarquia estructurada de tècniques, cadascuna amb el seu propi paper en un sistema d'empaquetament eficient.

L'enfocament més robust i generalment acceptat per a la detecció de col·lisions en escenes 3D complexes es basa en dues fases ben definides: una fase ampla (broad phase) i una fase estreta (narrow phase)¹. La fase ampla té l'objectiu de filtrar ràpidament parells d'objectes que són massa llunyans per col·lidir, mentre que la fase estreta realitza càlculs computacionalment costosos només per aquelles parelles que tenen probabilitat de col·lidir. En el context d'empaquetament, on es necessita comprovar cada nova peça contra totes les altres ja posicionades, aquest enfocament és essencial per mantenir la velocitat. La primera línia de defensa són els volums envolupants aïllats als eixos (Axis-Aligned Bounding Box, AABB). Un AABB és una caixa rectangular que envolta completament un model 3D, amb les seves cares paral·leles als eixos de coordenades^{1 35}. Comprovar si dos AABB col·lideixen és extremadament ràpid, ja que només requereix comparacions escalars entre els seus límits³⁵. Totes les biblioteques modernes de processament 3D, incloent-hi **trimesh** i **Open3D**, implementen funcionalitats per calcular i comparar AABB de manera eficient^{19 29}.

Tanmateix, per a geometries complexos i irregulars, un únic AABB pot ser molt gros i deixar molta "cendra" (espai buit), fent que les comprovacions de col·lisió siguin menys efectives. Una

opció millorada són les Caixes Orientades Mínimes (Minimum Oriented Bounding Boxes, OBB), que permeten que la caixa envoltant estigui orientada de forma òptima per reduir aquesta cendra. Tot i això, comprovar la col·lisió entre OBB és més complex que entre AABB. Una solució més sofisticada i eficient són les Jerarquies de Volums Envoltants (Bounding Volume Hierarchies, BVH) ^{1 34}. Una BVH construeix un arbre binari on cada node és un volum envoltant (sovint un AABB) que cobreix els volums dels seus fills. Aquesta estructura permet podar gran part de l'espai de cerca; si dos volums de nivell superior d'una BVH no col·lideixen, no cal examinar cap dels seus objectes fills ^{1 42}. El projecte **pack3d** demostra l'ús de precomputació de BVHs per accelerar dràsticament la detecció d'interseccions ²², i **trimesh.collision.mesh_to_BVH** ofereix aquesta capacitat directament ²⁷.

Per a models 3D perfectament estancs (watertight), **trimesh** ofereix una eina central: la classe **CollisionManager** ^{16 27}. Aquest gestor permet afegir múltiples malles i realitzar comprovacions de col·lisió de diversos tipus: * **in_collision_internal**: Comprova si la malla que conté el manager col·lideix amb ella mateixa. * **in_collision_other**: Comprova si la malla col·lideix amb qualsevol altra malla afegida al manager. * **in_collision_single**: Comprova si una nova malla col·lideix amb cap de les ja existents.

Aquests mètodes són idealment situats a la fase estreta de la detecció de col·lisions. Abans de cridar-los, però, sempre hauria de precedir una comprovació RÀPIDA amb AABB per descartar la majoria de casos. Per exemple, abans de fer **mesh1.check_collision(mesh2)**, es podria fer **mesh1.bounding_box_oriented.is_collision(mesh2.bounding_box_oriented)**.

Per a una aplicació d'empaquetament que ha de ser "precisa" i no "ràpida", la fase estreta no pot ser ignorada. Les opcions inclouen: * Intersecció Triangle-Triangle: El mètode més bàsic i exhaustiu, que consisteix a comprovar cada triangle de la primera mesh amb cada triangle de la segona. És computacionalment molt costós, especialment per a models amb centenars de milers de triangles ³⁷. * Algorismes Eficients: Biblioteques com **trimesh** utilitzen algorismes optimitzats com l'algorisme de Gilbert-Johnson-Keerthi (GJK) per a formes convèxes ^{34 39} o implementacions basades en predicats geomètrics exactes per evitar errors numèrics ¹². L'opció **findCollidingTriangles** de la llibreria MeshLib també és una referència ¹. * Voxelització: Aquesta tècnica converteix les malles en una matriu 3D de vòxels (pixels 3D). La detecció de col·lisió es redueix a comprovar si dos vòxels ocupats comparteixen la mateixa posició en la matriu ^{20 30}. Aquest mètode és relativament senzill de programar i la seva precisió es controla ajustant la resolució del voxel. **trimesh** ofereix suport per a la voxelització amb **mesh.voxelized()** ¹⁶.

En conclusió, per a la millora immediata de l'aplicació actual, es recomana substituir el bloc de codi que gestiona la posició de les peces per un nou bloc que segueixi aquest flux: 1. Generar Candidats: Crear una llista de possibles posicions per a la propera peça, potser mitjançant un algorisme de faristol (bin-packing heuristic). 2. Fase Amplia (Broad Phase): Per a cada candidat, crear una nova instància de la peça en aquella posició i obtenir la seva caixa envoltant (AABB). 3. Comparació Ràpida: Utilitzar una estructura de dades espacial (com un quadtree o un simple array) per guardar les caixes envolupants de totes les peces ja posicionades. Comparar l'AABB de la peça candidata amb aquestes per descartar immediatament les posicions que estan fora del rang. 4. Fase Estreta (Narrow Phase): Només per a les poques posicions que passen la fase ampla, realitzar una comprovació de col·lisió precisa amb **CollisionManager**. Si no hi ha col·lisió, la posició és vàlida.

Aquest doble nivell de comprovació assegura tant la precisió com una bona eficiència, sent el punt de partida per a qualsevol sistema d'empaquetament d'alt rendiment.

Gestió de Rotacions: Estratègies Sistemàtiques per a l'Optimització de l'Espai

Un altre defecte crítica de l'enfocament actual és la falta de prova sistemàtica de diferents rotacions per trobar l'orientació òptima per a cada peça dins del contenidor. L'algoritme intenta provar algunes rotacions ortogonals (múltiples de 90 graus), però aquesta implementació manual i limitada no es pot considerar una estratègia completa. Per a objectes irregulars, la millor orientació per maximitzar l'eficiència d'empaquetament sovint no és una alineació simple amb els eixos. Per tant, gestionar les rotacions de manera sistemàtica i eficient és crucial per tal de minimitzar l'espai buit.

Hi ha diverses maneres de representar i aplicar rotacions en tres dimensions, cadascuna amb les seves avantatges i inconvenients. L'enfocament més accessible per a un principiant en geometria 3D són els angles d'Euler, que descriuen una rotació com una seqüència de tres rotacions al voltant dels eixos X, Y i Z ¹¹. No obstant això, els angles d'Euler sovint presenten el problema del bloqueig cardànic (gimbal lock), on es perd un grau de llibertat, i la composició de rotacions no és intuïtiva ¹¹. Una alternativa més robusta i comuna en computació gràfica i robòtica són els quaternions ¹¹. Un quaternion és una extensió dels nombres complexos que pot representar una rotació 3D de manera compacta i evita el bloqueig cardànic ¹¹. La conversió entre quaternions i altres representacions com matrius de rotació o angles d'Euler és ben documentada ^{11 13}.

La implementació de la prova de rotacions es pot estructurar de dues maneres principals: exhaustiva o heurística. 1. Prova Exhaustiva: Aquest mètode implica crear una llista finita de rotacions possibles i provar-les totes. Això garanteix trobar la millor orientació si es combina amb una detecció de col·lisió precisa. Tanmateix, la quantitat de rotacions necessàries pot créixer ràpidament. *

Rotacions Ortogonals: Provar totes les combinacions de rotacions de 90 graus al voltant dels tres eixos resulta en 24 rotacions úniques per a un objecte genèric (incloent la rotació zero) ²².

L'algoritme **pack3d** fa servir precomputació sobre aquestes 24 rotacions per accelerar les comprovacions posteriors ²². L'algorisme KRIH per 2D irregular ho fa de manera similar, generant angles basats en la forma convexa de l'objecte ⁴³.

* Divisió Uniforme de l'Espai: Una aproximació més general seria dividir l'esfera unitat que representa totes les rotacions possibles en una graella regular d'angles d'Euler o quaternions. Per exemple, es podrien provar increments de 30 graus als tres eixos, tal com es fa en l'algorisme PackMerger ³.

2. Prova Heurística: Aquest mètode no prova totes les rotacions possibles, sinó que es concentra en un subconjunt prometededor. Això pot ser

significativament més ràpid, però no garanteix l'optimum global. * Basat en la Forma Convexa: Una heurística eficaç és analitzar l'envolupant convexa de l'objecte ³.

Identificant les cares planes de l'envolupant convexa, es pot determinar quines orientacions són "estables" o "significatives" per a l'empaquetament. Per exemple, es podrien provar només les orientacions que fan coincidir una de les cares planes de l'envolupant convexa amb el "terra" del contenidor. * Basat en l'Angle d'Euler

Intrínsec: Els angles d'Euler tenen una convenció intrínseca (rotacions respecte als nous eixos locals de l'objecte) que es pot utilitzar per definir un espai de rotació. L'algorisme de DeepPack, per exemple, explora l'espai de rotacions mitjançant moviments aleatoris en aquest espai ¹⁴.

L'enfocament híbrid proposat per a l'aplicació de l'usuari seria el més pragmàtic. La lògica d'empaquetament hauria de decidir, per a cada peça abans de posicionar-la, quin conjunt de rotacions provar. Això es pot implementar mitjançant una funció com `get_possible_rotations(mesh)` que retorni una llista de transformacions de rotació (per exemple, quaternions o matrius 4x4 de `trimesh`) a provar. Aquesta funció podria implementar una regla condicional:

```
def get_possible_rotations(mesh):  
    # Si l'objecte té un nombre baix de cares, probablement és simple  
    if len(mesh.faces) < 15:  
        # Potser només provar les 24 rotacions ortogonals  
        return generate_orthogonal_rotations()  
    else:  
        # Per a objectes complexos, provar una divisió uniforme de l'espai  
        return generate_uniform_rotation_grid(step_degrees=30)
```

Després, a l'algoritme principal d'empaquetament, es recorreria aquesta llista de rotacions per a cada peça candidata, aplicant cada una d'elles, executant la detecció de col·lisió i seleccionant la combinació (rotació + posició) que resulti en la millor puntuació segons l'heurística de l'algoritme (per exemple, la posició més baixa possible).

Finalment, es pot considerar l'optimització de l'ordre de les peces. Encara que l'usuari vol un procés totalment automàtic, l'ordre en què s'introdïsquen les peces afecta enormement l'eficiència final. L'algorisme de l'aplicació actual sembla tractar les peces en l'ordre en què es carreguen. No obstant això, estudis han demostrat que optimitzar l'ordre d'inserció pot ser tan important com l'algorisme d'empaquetament mateix ³. Algunes estratègies heurístiques per ordenar les peces abans de començar l'empaquetament inclouen: * Bigger-First: Empaqueta les peces més grans primer. Això omple l'espai major i crea cavitats menors per a les peces més petites. Aquesta heurística es pot trobar en llibreries com `py3dbp` ¹⁵. * Més Plans Primer: Ordena les peces segons la seva relació àrea/volum, preferint les que tenen una base més estable. * Ordenació Aleatòria: Alguns algorismes com `py3dbp` ofereixen una funció `shuffle_pack` que executa múltiples intents d'empaquetament amb un ordre aleatori de peces per buscar una millor solució ⁴⁷.

Implementar una heurística d'ordenació inicial, com ara "Bigger-First", juntament amb una prova exhaustiva de rotacions ortogonals per a peces simples i una prova heurística per a les complexes, proporcionaria un punt de partida potent i fàcil d'implementar.

Selecció i Implementació d'Algoritmes d'Empaquetament 3D Bin Packing

Amb una detecció de col·lisió robusta i una gestió de rotacions sistemàtica, l'etapa següent és triar i implementar un algorisme d'empaquetament 3D bin packing. L'enfocament actual, que simplement omple una graella, no és adequat per a formes irregulars. Els algorismes de la literatura acadèmica i professional es poden classificar en heurístiques, metaheurístiques i mètodes basats en machine learning. Per a una implementació que prioritzi la precisió sobre la velocitat màxima, les heurístiques i les metaheurístiques són l'opció més realista i accessible.

Les heurístiques són regles simples i eficients que construeixen una solució de forma iterativa. Moltes d'aquestes es basen en conceptes inspirats en la física o l'intuïció humana. L'algorisme més citat en el context de l'usuari és el Deepest Bottom Left with Fill (DBLF), introduït per Korhan Karabulut i Mustafa Murat Inceoğlu ^{26 45}. Aquesta heurística funciona iterativament: 1. Selecciona una Posició: Troba el "forat" més profunda (el que té el z més petit) en una estructura de dades que representa l'espai buit del contenidor. 2. Selecciona una Peça: Triar una peça de l'entrada. 3. Col · loca la Peça: Intenta col · locar la peça en aquest forat. Això implica provar diferents rotacions i desplaçaments. 4. Actualitza l'Espai: Si la col · locació és vàlida (sense col · lisió), es treu l'espai ocupat per la peça del conjunt d'espais buits i es rellenen els nous forats creats.

Variant de l'algorisme DBLF, l'heurística Bottom-Left-Fill (BLF) busca el punt més avall i a l'esquerra on es pot col · locar una peça ^{2 40}. L'algorisme **packmerger** fa servir un mètode basat en camps d'alçada (height field – based packing) inspirat en aquesta idea ³. Aquests algorismes són fàcils de conceptualitzar i implementar, i serveixen com a excel · lent base per a un sistema d'empaquetament.

Les metaheurístiques són estratègies més generals que guien una cerca heurística per explorar l'espai de solucions de manera més eficient que una cerca exhaustiva. Alguns exemples citats en la literatura relevant són: * Recuit Simulat (Simulated Annealing): Aquesta tècnica, utilitzada per a problemes d'empaquetament de caixes tractables ⁹, permet acceptar solucions pitjors temporalment per sortir de mínims locals i trobar una solució global millor. Es pot adaptar per a models irregulars. * Cerca Local Iterada (Iterated Local Search, ILS): S'utilitza per a problemes d'empaquetament d'objectes irregulars, modificant iterativament una solució actual per buscar-ne una millor ²⁰. * Cerca Tabú (Tabu Search, TS): S'utilitza per a optimitzar l'ordre d'empaquetament, recordant les decisions recent seves per evitar cicles ^{3 20}. * Algorismes Genètics (Genetic Algorithms, GA): Es combinen amb DBLF per millorar-ne el rendiment en problemes de vehicle routing ²⁶.

Finalment, les aproximacions basades en aprenentatge per reforç (Reinforcement Learning, RL) han mostrat resultats prometedors. Models com DeepPack ¹⁴ o OnlineBPH ^{4 45} entren agents per aprendre l'estratègia d'empaquetament òptima. Tot i que aquestes solucions poden assolir altes taxes d'ocupació ⁴⁵, sovint requereixen temps d'entrenament massiu i accés a paquets de desenvolupament complements (com PyTorch) ⁴, i sovint són per a fins acadèmics ⁴.

Donat que l'objectiu és una solució simple i funcional, es recomana iniciar amb una implementació de l'algorisme DBLF. A continuació es presenta un esqueletró d'implementació en pseudocodi clar que integra les idees discutides:

```
ALGORISME pack_3d(contenidor, llista_de_pieces):  
    # Inicialitzar el gestor de col·lisions  
    collision_manager = CollisionManager()  
    # Afegir el contenidor com a objectes immobles  
    collision_manager.add_mesh(name="container", mesh=contenidor.mesh)  
  
    # Estructura per emmagatzemar l'espai buit restant (forats)  
    # Podria ser una llista de caixes 3D (bounding boxes)  
    espais_buits = [contenidor.bounds]  
  
    # Ordenar les peces per volum (heurística Bigger-First)  
    llista_de_pieces.sort(key=lambda x: x.volume, reverse=True)
```

```

# Crear una llista per emmagatzemar les peces col·locades
peces_col·locades = []

PER CADA peça IN llista_de_pieces:
    # Generar el conjunt de rotacions a provar per aquesta peça
    rotacions_per_provar = get_possible_rotations(peça.mesh)

    millor_posicio = None
    millor_rotacio = None

    PER CADA rotacio IN rotacions_per_provar:
        # Aplicar la rotació provisional
        peça_temporal = apply_rotation(peça.mesh, rotacio)

        # Generar candidats de posició (ex: faristol bottom-left-fill)
        candidats_de_posicio = generar_candidats_bottom_left(espais_buits, peça_temporal)

        PER CADA candidat IN candidats_de_posicio:
            # Calcular la posició absoluta de la peça
            posicio_absoluta = calculate_absolute_position(candidat, mesh)

            # Crear una nova instància de la peça a la posició provisional
            peça_prova = create_transformed_mesh(peça_temporal, posicio_absoluta)

            # Comprovar col·lisió amb el gestor de col·lisions
            SI NO collision_manager.in_collision_with(peça_prova):
                # Aquesta és una posició vàlida! Guardem-la
                millor_posicio = posicio_absoluta
                millor_rotacio = rotacio
            TREU DE LA BÚSQUEDA

# Si s'ha trobat una posició vàlida
IF millor_posicio I millor_rotacio:
    # Aplicar la rotació i la posició finals
    peça.mesh = apply_rotation(peça.mesh, millor_rotacio)
    peça.position = millor_posicio

    # Afegir la peça al gestor de col·lisions
    collision_manager.add_mesh(name=peça.id, mesh=peça.mesh)

    # Actualitzar la llista d'espais buits eliminant l'espai ocupat
    espais_buits = update_free_spaces(espais_buits, peça)

    # Afegir a la llista de peces col·locades
    peces_col·locades.append(peça)

RETURN peces_col·locades, llista_de_pieces_no_col·locades

```

Aquest pseudocodi il·lustra com es poden integrar totes les parts: detecció de col·lisió (`collision_manager`), gestió de rotacions (`get_possible_rotations`), i l'heurística

principal (DBLF). Aquesta estructura modular facilita la prova i l'iteració sobre cada component independentment.

Integració de Biblioteques Python per a l'Empaquetament 3D

La tria de les eines de codi obert adequades pot accelerar significativament el desenvolupament i assegurar la robustesa de l'aplicació. L'aplicació actual ja utilitza **trimesh** per a la càrrega de models i càlculs matemàtics, una elecció excel·lent que es mantindrà com a nucli de la solució. El nostre objectiu és identificar i integrar biblioteques complementàries que afrontin les debilitats específiques identificades: detecció de col·lisions, algorismes d'empaquetament i, en menor mesura, processament de models 3D.

La taula següent resumeix les biblioteques clau i la seva contribució al problema:

Biblioteca	Versió Principal	Contribució Clau	Integració Amb Trimesh	Nota Adicional
trimesh	4.7.4 ¹⁹	Càrrega de models (STL/ OBJ), càlculs geomètrics, CollisionManager ¹⁶ , operacions de transformació.	Base central de l'aplicació actual.	Necessita dependències opcionals com pyglet per a la visualització ¹⁹ .
py3dbp	1.1.2 ¹⁵	Implementació d'empaquetament 3D per a caixes rectangulars (bin packing).	No, treballa amb caixes, no amb malles 3D.	Pot servir com a base per entendre el patró de classes (Bin/Item/ Packer) ¹⁷ .
Open3D	-	Processament 3D robust, càrrega de malles, normals, detecció de col·lisió (possiblement).	Sí, es pot convertir entre trimesh i Open3D ⁴⁶ .	Més orientat al processament de núvols de punts i meshes robusta, no tant a l'heurística d'empaquetament ²⁸ .
numpy	-	Càlculs matemàtics i manipulació de dades arrays.	Ja s'utilitza en l'aplicació actual.	Fonamental per a les operacions de transformació i càlculs vectorials.
scipy	-	Tools científics, KDTree per proximitat i voxelització.	Sí, útil per augmentar la precisió de trimesh ³⁰ .	KDTree pot ajudar a cercar vèrtexs propers durant la voxelització ³⁰ .
pack3d	-	Pre-computació de BVH per acceleració de col·lisions.	Sí, es pot integrar la	El codi és un bon recurs per veure

Biblioteca	Versió Principal	Contribució Clau	Integració Amb Trimesh	Nota Adicional
			lògica de pre-computació.	tècniques d'acceleració ²² .

Com ja s'ha mencionat, **trimesh** ja ofereix la funció **CollisionManager**, que s'ha d'integrar com a motor de detecció de col·lisió central. La seva eficàcia es pot amplificar combinant-la amb una estratègia de filtre de parelles d'objectes basada en AABB, que **trimesh** també calcula ràpidament.

La llibreria **py3dbp** és particularment interessant perquè, encara que està dissenyada per a caixes, el seu patró de programació i la seva heurística bàsica són molt educatius ^{17 24}. El seu repositori GitHub i el notebook de Google Colab mostren clarament com es defineixen **Bin** (contenidors) i **Item** (peces), es crea un **Packer** i es crida el mètode **pack()** ^{17 25}. Aquest patró es pot adaptar fàcilment per a models 3D: * En comptes de crear un objecte **Item** amb amplada, alçada i profunditat, es podria crear un objecte **MeshItem** que accepti una instància de **trimesh.Mesh**. * En comptes de fer servir les dimensions de la caixa envolvent per a la inserció, l'algoritme **py3dbp** podria ser modificant per utilitzar l'àrea de la cara inferior de la peça (calculada a partir de la seva envolupant convexa) com a criteri per a l'heurística "Bigger-First" ².

L'ús de **Open3D** podria ser beneficiós si l'aplicació necessités funcions de processament de models més avançades, com la neteja de malles, el rebalancejament de l'estructura de dades per a la cerca o la renderització interactiva més ràpida. Tot i que la integració amb **trimesh** es pot fer mitjançant funcions auxiliars com **mesh_to_trimesh** ⁴⁶, en molts casos **trimesh** ja ofereix una gamma completa de funcions que cobreixen les necessitats bàsiques d'un sistema d'empaquetament ^{19 28}.

És important destacar que alguns elements de la informació proporcionada poden portar a errors. Per exemple, es cita **py3dbp** com a capaç de gestionar fitxers STL/OBJ, però aquesta afirmació es basa en una font que es refereix a una versió anterior (0.3) que probablement no estava completa ²⁴. La documentació i el codi actual de **py3dbp** mostren que opera exclusivament sobre caixes definides per dimensions numèriques ^{15 47}. És per això que es conclou que **py3dbp** no pot substituir la necessitat de processar malles 3D amb **trimesh**.

En resum, la recerca recomana una estratègia d'integració centrada en **trimesh** com a motor geomètric principal, augmentat amb: 1. Estructures de Dades Espacials: Per accelerar la detecció de col·lisions. Això pot implementar-se manualment o mitjançant funcions de **trimesh** com **mesh_to_BVH** ²⁷. 2. Patrons d'Aprenentatge: Pel patró de classe **Packer/Bin/Item** de **py3dbp** per organitzar el codi ¹⁷. 3. Funcions de Suport: De biblioteques com **scipy** per a tasques com la voxelització o la proximitat ³⁰.

Aquest enfocament modular permet construir una solució robusta i escalable sense necessitat de dependències massives o de sistemes monolítics.

Alternatives i Solucions Avançades: Des de Biblioteques Externes a l'Aprenentatge per Reforç

Encara que l'objectiu principal és desenvolupar una solució autònoma i personalitzada en Python, és útil considerar alternatives i components d'altres sistemes que poden inspirar o simplificar el procés. Aquesta secció analitza l'ús de slicers d'impressió 3D com a API, frameworks externs i mètodes d'aprenentatge per reforçament (RL).

L'ús de slicers com PrusaSlicer o Cura com a motors d'empaquetament via API ha estat una opció popular. Els slicers estan dissenyats per a l'empaquetament de models 3D per a l'impressió, per tant, la seva lògica interna sembla idònia. No obstant això, la recerca no ha pogut confirmar l'existència d'API públiques per a aquests slicers que permetin a una aplicació externa carregar models, especificar un contenidor i rebre la disposició resultant. La llibreria **curaengine** (part de la suite de Cura) sí que ofereix una API, però està dissenyada per a un ús intern i no per a la interoperabilitat amb altres aplicacions ²⁸. El repositori **octoprint** esmenta que **curaengine** pot ser usat com a llibreria, però no s'ofereix una guia detallada ²⁸. Per tant, l'ús directe d'un slicer com a API externa sembla més teòric que pràctic en aquest moment.

Una altra àrea interessant són els marcs i projectes de recerca que ofereixen solucions més sofisticades. PackMerger és un marc de codi obert escrit en C++ que ofereix una solució integral per a l'optimització d'empaquetament per a la fabricació additiva ^{3 6}. El seu procés inclou la conversió de la malla a una closca buida, la segmentació i, finalment, l'empaquetament òptim utilitzant una combinació de tècniques, incloent-hi la cerca tabú per a l'optimització de l'ordre. Encara que està escrit en C++, el seu paper de recerca detalla detalladament les seves etapes i algorismes, que poden servir com a guia per a una implementació en Python ³. Altres projectes com **Online-3D-BPP-PCT** i **Online-3D-BPP-DRL** de alexfrom0815 mostren implementacions de deep reinforcement learning per a l'empaquetament 3D en línia, que són força avançades i estan destinades a entorns de recerca ^{4 21}.

Les solucions basades en aprenentatge per reforçament (RL) han demostrat ser altament efectives, però comporten una barreja de dificultat i recursos. Models com DeepPack, TAP-Net o HHPPO han aconseguit resultats notables en termes de densitat d'empaquetament, a vegades superant els algorismes heurístics tradicionals ^{8 14 41}. No obstant això, aquestes solucions tenen uns requisits significatius: * Entrenament: Requereixen un temps considerable d'entrenament (hores o dies) en grans conjunts de dades per aprendre l'estratègia d'empaquetament ^{4 14}. * Complexitat: Impliquen el maneig de frameworks de machine learning com PyTorch, la definició d'arquitectures de xarxes neuronals i la configuració de funcions de recompensa ⁴. * Restriccions: Moltes d'aquestes implementacions són per a fins acadèmics i poden tenir restriccions sobre el nombre d'ítems o la complexitat dels models ^{4 14}.

Donat que l'objectiu de l'usuari és una solució "simple i funcional" amb un compromís acceptable entre precisió i velocitat, es desaconsella l'ús d'aprenentatge per reforçament per al seu cas d'ús immediat. El cost i la complexitat superen amb prou feines els beneficis esperats per a una única aplicació personal.

En canvi, una estratègia més pragmàtica consisteix a adoptar components d'aquests sistemes avançats. Per exemple, l'heurística DBLF i la seva variació BLBF són l'element central de molts sistemes eficients ^{26 45}. L'ús d'una BVH per accelerar la detecció de col·lisions, com es fa a **pack3d** ²², és una millora estàndard que es pot implementar independentment. La incorporació d'una estructura d'espai buit per gestionar on es poden col·locar les peces, en lloc d'una simple graella, és una altra característica que pot incrementar significativament l'eficiència ²⁶.

Per tant, en lloc de buscar una "caixa negra" que faci tot el treball, es recomana adoptar un enfocament híbrid: 1. Construir una Base Heurística Robusta: Implementar un algorisme DBLF o BLF basat en **trimesh** com s'ha suggerit abans. 2. Incorporar Tècniques d'Acceleració: Integrar una estructura de dades basada en BVH per a la detecció de col·lisions ²² i una estructura d'espai buit per a la selecció de candidats ²⁶. 3. Considerar Optimitzacions Post-Empaquetament: Implementar una fase de "recollida" (packing phase) seguida d'una fase d'optimització (optimization phase), inspirada en marcs com **packmerger** ³. Aquesta fase podria moure peces lleugeres cap a llocs vacants, girar-les per millorar l'estabilitat o aplicar una simulació de dinàmica de cossos rígids per "ajuntar" les peces col·locades ³⁹.

Aquest enfocament permet construir una solució potent i eficient utilitzant components i idees comprovades de la recerca avançada, sense haver de sumergir-se en el camp complex de l'aprenentatge per reforçament.

Recomanacions Finales i Ruta de Desenvolupament

Després d'una anàlisi exhaustiva dels problemes actuals i de les solucions disponibles, es poden formular un conjunt de recomanacions pràctiques i una ruta de desenvolupament clar per a l'usuari. L'objectiu final és transformar l'aplicació actual, amb el seu algorisme deficient, en un sistema robust, precís i automàtic per a l'empaquetament 3D d'objectes irregulars.

Recomanació Central: Adoptar una Arquitectura Modular Basada en Heurística

Es desaconsella fortament continuar amb l'enfocament actual de la graella regular. Els seus fracassos en la detecció de col·lisions i la gestió de rotacions són fonamentals i insuperables sense una refactorització completa. La recerca conclou que la millor trajectòria és adoptar una arquitectura modular que integri quatre components clau:

1. Motor Geomètric i de Col·lisions: Centrat en la llibreria **trimesh**. Aquesta biblioteca ja és coneguda per l'usuari i és la base ideal per a càlculs precisos. La seva classe **CollisionManager** ha de ser el nucli per a totes les comprovacions de sobrepassament ^{16 27}. Aquest component s'encarregarà de validar que cap peça col·lideix amb cap altra o amb les parets del contenidor abans de considerar una posició com a vàlida.
2. Gestor de Rotacions Sistemàtic: Aquest component s'hauria de separar de l'algorisme principal. La seva funció seria, donat un model 3D, retornar una llista de transformacions de rotació (matrius 4x4 o quaternions) a provar. L'algorisme hauria de distingir entre peces "simples" (menys de 15 cares) i "complexes". Per a les simples, es poden provar les 24 rotacions ortogonals (múltiples de 90°) ²². Per a les complexes, es poden provar una mostra més ampla de l'espai de rotació o utilitzar una heurística basada en l'envolupant convexa ³.

3. Algorisme d'Empaquetament Principal: L'algorisme hauria de ser una implementació de l'heurística Deepest Bottom Left with Fill (DBLF) ^{26 45}. Aquesta heurística és un punt de partida potent perquè:

- És relativament fàcil d'entendre i implementar.
- Construeix la solució de manera constructiva, col · locant una peça a la vegada.
- La seva fase "Fill" ajuda a omplir cavitats i aconseguir una distribució més compacta.
- Pot ser fàcilment augmentada amb les estructures de dades i acceleradors mencionats.

4. Estructura d'Espai Buit: Aquest component és la clau per a l'eficiència de l'algorisme DBLF. En lloc d'una graella fixa, s'hauria d'utilitzar una estructura de dades (per exemple, una llista o un arbre) que representi l'espai lliure del contenidor. Quan es col · loca una peça, aquesta estructura s'actualitza per a reflectir l'espai nou creat. Aquesta tècnica es pot trobar en variants com l'ús d'arbres RTree per gestionar l'espai lliure ²⁶.

Ruta de Desenvolupament Passo a Passo

Seguint aquesta arquitectura, es pot planificar el desenvolupament en etapes progressives:

Etapa 1: Refactorització i Validació * Substituir completament la funció `_fallback_optimization` i `_simple_grid_packing`. * Implementar una funció `check_position_valid(mesh, position, rotation, collision_manager, placed_items)` que utilitzi el `CollisionManager` de `trimesh` per verificar si una peça pot anar en una posició donada sense col · lisions. * Provar aquesta funció amb diverses combinacions de posició i rotació per garantir la seva precisió.

Etapa 2: Implementació de la Gama de Rotacions * Crear la funció `get_possible_rotations(mesh)`. * Provar aquesta funció per assegurar-se que retorna el conjunt desitjat de rotacions (ex: 24 ortogonals per a peces simples).

Etapa 3: Implementació de l'Heurística DBLF Bàsica * Implementar una versió inicial de l'algorisme DBLF. * En aquesta etapa, es pot començar amb una implementació simple de "bottom-left" sense el "fill" complet, per exemple, buscant el punt més a l'esquerra i més avall de l'espai lliure disponible. * Provar l'algorisme amb models simples (ex: un triangle rectangle) per verificar que aconsegueix una col · locació coherent.

Etapa 4: Millora i Acceleració * Integrar l'estructura d'espai buit per gestionar on es poden col · locar les peces de manera més eficient que amb una simple cerca de graella. * Integrar una estructura d'acceleració com una Bounding Volume Hierarchy (BVH) per a la detecció de col · lisions. Això millorarà dràsticament el rendiment, especialment amb models complexos ^{1 22}. * Afegir la fase "Fill" de l'algorisme DBLF, que consisteix a intentar col · locar la peça en cada un dels forats creats pel seu propi posicionament.

Etapa 5: Optimització i Resultats * Afegir una fase post-procesament per intentar millorar la solució obtinguda. Això podria incloure moure peces lleugeres cap a cavitats, provar rotacions addicionals per a les peces ja posades o simular una sacseuada del contenidor per compactar les peces ³⁹. * Implementar una heurística d'ordenació inicial, com "Bigger-First", per influir positivament en el resultat final ¹⁵.

En conclusió, el camí cap a una solució d'empaquetament 3D robusta i eficient no resideix en la millora d'un algorisme deficient, sinó en la seva substitució per un sistema modular i ben dissenyat. Centrant-se en la validació de col·lisions amb **trimesh**, la gestió sistemàtica de rotacions i l'implementació d'una heurística com DBLF amb estructures d'acceleració, l'usuari pot construir una aplicació que sigui a la vegada preciosa, fiable i satisfactoria per al seu ús.

en

1. 3D Collision Detection Library for Python and C++ - MeshLib <https://meshlib.io/feature/collision-detection/>
2. [PDF] A Constructive Heuristic Algorithm for 3D Bin Packing of Irregular ... <https://arxiv.org/pdf/2206.15116>
3. [PDF] PackMerger: A 3D Print Volume Optimizer | Computer Graphics Forum https://www.cs.purdue.edu/cgvlab/www/resources/papers/Vanek-Computer_Graphics_Forum-2014-PackMerger_A_3D_Print_Volume_Optimizer.pdf
4. GitHub - alexfrom0815/Online-3D-BPP-PCT <https://github.com/alexfrom0815/Online-3D-BPP-PCT>
5. Seamless 3D Printing API to Streamline your Workflow - 3DPrinterOS <https://www.3dprinter.com/3d-printing-management-apis-reference>
6. PackMerger: A 3D Print Volume Optimizer - Wiley Online Library <https://onlinelibrary.wiley.com/doi/abs/10.1111/cgf.12353>
7. Learning with 3D rotations, a hitchhiker's guide to SO(3) - arXiv <https://arxiv.org/html/2404.11735v1>
8. A Multi-Heuristic Algorithm for Multi-Container 3-D Bin Packing ... https://www.researchgate.net/publication/379084031_A_Multi-Heuristic_Algorithm_for_Multi-container_3-D_Bin_Packing_Problem_Optimization_using_Real_World_Constraints
9. [PDF] The 3D-Packing by Meta Data Structure and Packing Heuristics <https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=0343bd1dfa565a98363487b5a9bed1d6018376c4>
10. Carvable packing of revolved 3D objects for subtractive manufacturing <https://www.sciencedirect.com/science/article/pii/S1524070325000293>
11. Rotation formalisms in three dimensions - Wikipedia https://en.wikipedia.org/wiki/Rotation_formalisms_in_three_dimensions
12. [PDF] Efficient Geometrically Exact Continuous Collision Detection <https://www.cs.ubc.ca/labs/imager/tr/2012/ExactContinuousCollisionDetection/beb2012.pdf>
13. Combining Two 3D Rotations - Mathematics Stack Exchange <https://math.stackexchange.com/questions/22437/combining-two-3d-rotations>

14. Python example of 3D bin packing problem and visualization <https://stackoverflow.com/questions/68953770/python-example-of-3d-bin-packing-problem-and-visualization>
15. Putting boxes in boxes: from knapsack to 3D bin packing <https://supplychaindatascientist.com.wordpress.com/2020/12/19/putting-boxes-in-boxes-from-knapsack-to-3d-bin-packing/>
16. trimesh 4.7.4 documentation <https://trimesh.org/trimesh.html>
17. enzoruiz/3dbinpacking: A python library for 3D Bin Packing - GitHub <https://github.com/enzoruiz/3dbinpacking>
18. An effective data structure for a 3D printing slicer API - IEEE Xplore <https://ieeexplore.ieee.org/document/7804739/>
19. mikedh/trimesh: Python library for loading and using ... - GitHub <https://github.com/mikedh/trimesh>
20. Voxel-Based Solution Approaches to the Three-Dimensional ... <https://pubsonline.informs.org/doi/10.1287/opre.2022.2260>
21. bin-packing · GitHub Topics <https://github.com/topics/bin-packing?l=python&utf8=%E2%9C%93>
22. Show HN: 3D Packing for 3D Printing | Hacker News <https://news.ycombinator.com/item?id=14614304>
23. The 3D bin packing problem for multiple boxes and irregular items ... <https://link.springer.com/article/10.1007/s10489-023-04604-6>
24. py3dbp · PyPI <https://pypi.org/project/py3dbp/0.3/>
25. [Baseline] 3D-bin packing with py3dbp - Kaggle <https://www.kaggle.com/howeverforever/baseline-3d-bin-packing-with-py3dbp>
26. A Hybrid Genetic Algorithm for Packing in 3D with Deepest Bottom ... https://www.researchgate.net/publication/225643879_A_Hybrid_Genetic_Algorithm_for_Packing_in_3D_with_Deepeset_Bottom_Left_with_Fill_Method
27. trimesh.collision - trimesh 4.7.4 documentation <https://trimesh.org/trimesh.collision.html>
28. Python Libraries for Mesh, Point Cloud, and Data Visualization (Part 1) <https://towardsdatascience.com/python-libraries-for-mesh-and-point-cloud-visualization-part-1-daa2af36de30/>
29. trimesh vs open3d vs volmdlr | OpenText Core SCA - Debricked <https://debricked.com/select/compare/pypi-volmdlr-vs-pypi-open3d-vs-pypi-trimesh>
30. How to Voxelize Meshes and Point Clouds in Python - Medium <https://medium.com/data-science/how-to-voxelize-meshes-and-point-clouds-in-python-ca94d403f81d>
31. Load the same .obj file by using Open3D and Trimesh respectively ... <https://stackoverflow.com/questions/75963475/load-the-same-obj-file-by-using-open3d-and-trimesh-respectively-leading-to-diff>

32. A new approach for bin packing problem using knowledge reuse ... <https://www.nature.com/articles/s41598-024-81749-5>
33. Optimize Your 3D Printing with Advanced 3D Printer Slicer Software <https://www.3dprinteros.com/3d-printer-management/advanced-3d-printer-slicer-software-optimizing-your-3d-printing-experience>
34. What are the commonly used collision detection techniques in 3D ... <https://www.tencentcloud.com/techpedia/100407>
35. 3D collision detection - MDN - Mozilla https://developer.mozilla.org/en-US/docs/Games/Techniques/3D_collision_detection
36. c++ - Collision detection between transformed meshes (not primitives) <https://gamedev.stackexchange.com/questions/198108/collision-detection-between-transformed-meshes-not-primitives>
37. Fast 3d Mesh Collision Detection - Math and Physics - GameDev.net <https://www.gamedev.net/forums/topic/646717-fast-3d-mesh-collision-detection/>
38. java - 3D Collision Mesh (more efficient collision calculation) <https://stackoverflow.com/questions/40100632/3d-collision-mesh-more-efficient-collision-calculation>
39. Dynamics simulation-based packing of irregular 3D objects <https://www.sciencedirect.com/science/article/abs/pii/S0097849324001316>
40. A New Bottom-Left-Fill Heuristic Algorithm for the Two-Dimensional ... <https://pubsonline.informs.org/doi/10.1287/opre.1060.0293>
41. Integrating Heuristic Methods with Deep Reinforcement Learning for ... <https://pmc.ncbi.nlm.nih.gov/articles/PMC11358981/>
42. Efficient collision detection using hybrid medial axis transform and ... <https://www.sciencedirect.com/science/article/pii/S1524070323000103>
43. [PDF] An Efficient Pixel-based Packing Algorithm for Additive ... https://optimization-online.org/wp-content/uploads/2022/08/An-Efficient-Pixel_based-Packing-Algorithm-for-Additive-Manufacturing-Production-Planning.pdf
44. Optimizing Space: How Python Revolutionizes Packing Furniture for ... <https://medium.com/@devin.richard.smith/optimizing-space-how-python-revolutionizes-packing-furniture-for-storage-and-moving-b586d7b494d2>
45. Machine Learning for the Multi-Dimensional Bin Packing Problem <https://arxiv.org/html/2312.08103v1>
46. geomapi.utils.geometryutils - GitLab KU Leuven <https://geomatics.pages.gitlab.kuleuven.be/research-projects/geomapi/geomapi/geomapi.utils.geometryutils.html>
47. Bin-Packing Add-on — ezdxf 1.4.2 documentation <https://ezdxf.readthedocs.io/en/stable/addons/binpacking.html>